

实验二: 井字棋与对抗搜索

布置时间: 2026/4/2

提交截止时间: 2026/4/16 11:59pm

一、4x4 改进版井字棋：

1. 问题描述:

通过 AI 辅助编程实现一个 4x4 井字棋 (Tic-Tac-Toe) 的博弈算法与交互程序。棋盘为 4x4 的方格，人与AI轮流在空白格子中放置自己的标记（玩家为X，AI为O）。率先在横线、竖线或斜线上将 4个自己的标记连成一线的玩家获胜；若棋盘填满且无人连成 4 颗，则判为平局。每一局随机选定一名玩家为先手，但由于井字棋中存在先手必胜策略，因此改进版加入了pie规则，规则说明为：

- **第一步：** 玩家 A（被选为先手）在棋盘上的任意位置落下第一颗棋子。
- **第二步（核心）：** 玩家 B（后手）在看到这颗棋子后，面临一个二选一的抉择：
 - 选择不交换：B 觉得 A 下的这步棋一般般，于是 B 拿出自己的棋子正常下第二步，游戏继续。
 - 选择交换 (Swap)：B 觉得 A 下的这步棋太完美、太有杀伤力了。于是 B 喊一声“我要交换！”，B 直接把 A 刚才下的那颗棋子据为己有（互换阵营）。此时，这颗棋子变成了 B 的第一步，而 A 被迫变成了后手，要去下全盘的第二步

代码的具体要求如下：

- (1) **基础 Minimax 与代码 Debug：** 学在浙大已上传一份包含错误的 Minimax 经典3*3井字棋半成品代码。你需要排错并跑通代码：
 - 错误1：找出并修复代码中潜藏的逻辑错误（代码能运行但不报错，但 AI 走棋逻辑愚蠢或违反博弈规则）
 - 错误2：解决抛出异常的系统报错（锻炼阅读堆栈日志进行 Debug 的能力）代码保存为debug.py。
- (2) **实现 Alpha-Beta 剪枝：** 在完全修复的 Minimax 代码基础上，将棋盘扩大成4*4并引入 Alpha-Beta 剪枝算法，大幅优化搜索树的遍历效率。
- (3) **实现深度受限的 Alpha-Beta 剪枝：** 由于 4x4 棋盘的状态空间（相比 3x3）呈指数级激增，所以需要为算法增加最大搜索深度限制（Depth-limited）
- (4) **设计启发式函数：** 配合深度受限搜索，设计并实现一个能够评估非终局面优劣的启发式函数（Heuristic Function），指导 AI 在算不到终点时做出最优局部决策
- (5) **UI 界面开发：** 设计并实现一个友好的可视化用户交互界面（GUI）或高质量的命令行交互界面，支持完整的人机对战流程。 (2) (3) (4) (5) 的功能均实现后将代码保存为 alpha_beta.py

2. 提交要求:

- (1) 程序源码（需包含 debug.py, alpha_beta.py）；
- (2) 实验报告中包含
 - 与 AI 交互的过程(包括提示词、排错等);
 - 核心代码的理解与注释（特别是 Alpha-Beta 剪枝的边界传递逻辑、深度受限模块的代码说明）；
 - 启发式函数的设计（用清晰的逻辑或数学公式说明你是如何量化不同盘面优势的）；
 - UI 界面设计展示（界面交互说明及对战运行效果截图）
 - 可选项：在同一残局下，对比使用深度限制/无限制Minimax 的思考耗时及评估节点数；
- (3) 所有内容打包成压缩包后上传学在浙大,压缩包命名格式为学号-姓名-实验 2